



SOFTEx
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO



TypeScript



Inicializar o projeto e instalar o TypeScript

`npm init -y`

- **O que faz:** cria um arquivo `package.json` na pasta atual.
- **Por que importa:** o `package.json` guarda informações do projeto (dependências, scripts, nome, versão). É o “cartão de identidade” do seu projeto Node.
- **Resultado esperado:** aparece um `package.json` na pasta.



Typescript



`npm install --save-dev typescript`

- **O que faz:** instala o **TypeScript** localmente (só para este projeto).
- **Por que importa:** o compilador `tsc` vem do pacote `typescript`. Usamos `--save-dev` porque é ferramenta de desenvolvimento (não é usada em produção).
- **Resultado esperado:** uma pasta `node_modules` e o `typescript` listado em "`devDependencies`" no `package.json`.



Typescript



`npx tsc --init`

- **O que faz:** cria o `tsconfig.json` com configurações do compilador.
- **Por que importa:** o `tsconfig.json` diz ao TypeScript **de onde** pegar os `.ts` e **para onde** gerar os `.js`, além das regras de compilação.
- **Resultado esperado:** arquivo `tsconfig.json` criado.



Typescript

Ajustar o `tsconfig.json`

```
{  
  
  {  
    "compilerOptions": {  
      "target": "ES2020",  
      "module": "commonjs",  
      "rootDir": "./src",  
      "outDir": "./dist",  
      "strict": true  
    }  
  }  
}
```





Typescript

- **target: "ES2020"** – escolhe “o nível” de JavaScript gerado (recursos disponíveis). ES2020 funciona bem no Node moderno.
- **module: "commonjs"** – formato de módulos do Node (o antigo **require**). Serve para garantir compatibilidade simples.
- **rootDir: "./src"** – onde estão os seus arquivos **.ts** (código-fonte).
- **outDir: "./dist"** – para onde vão os **.js** gerados (código compilado).
- **strict: true** – ativa regras de tipagem mais rígidas (pega mais erros cedo).

Dica: se quiser, pode adicionar **"include": ["src"]** para deixar explícito que só compila o que está em **src/**.





Typescript



Criar as pastas e o arquivo TypeScript

`mkdir src`

- O que faz: cria a pasta `src` (onde ficarão os `.ts`).

{



Typescript

Criar as pastas e o arquivo TypeScript

`src/index1.ts`



ts

 Copiar código

```
class Pessoa {  
  nome: string;  
  idade: number;  
  
  constructor(nome: string, idade: number) {  
    this.nome = nome;  
    this.idade = idade;  
  }  
  
  apresentar(): string {  
    return `Olá! Meu nome é ${this.nome} e ter  
  }  
}  
  
const aluno1 = new Pessoa("Ana", 20);  
console.log(aluno1.apresentar());
```





Typescript



- **O que há de TypeScript aqui:** anotações de tipo (`string`, `number`) e retorno do método (`: string`).
- **Importante:** as **anotações de tipo somem** na hora de compilar; servem só para o compilador te ajudar.



Typescript



Compilar o código

`npx tsc`

- **O que faz:** lê o `tsconfig.json`, pega todos os `.ts` de `src/` e gera `.js` em `dist/`.
- **Resultado esperado:** um arquivo `dist/index1.js` (com o mesmo “nome base” do `.ts`).



Typescript



Conferência rápida:

```
ls src    # deve listar index1.ts
```

```
ls dist   # deve listar index1.js depois de  
compilar
```



Typescript



Executar com Node

`node dist/index1.js`

- **O que faz:** roda o **JavaScript compilado** no Node (o Node **não executa** `.ts`).
- **Saída esperada:**

`Olá! Meu nome é Ana e tenho 20 anos.`



TypeScript



Exercicio 1)

1. Inicializar o projeto e instalar TypeScript

No terminal (dentro de **Exercicio 1**):

```
npm init -y
```

```
npm install --save-dev typescript
```

```
npx tsc --init
```

Isso cria os arquivos **package.json** e **tsconfig.json**.



Typescript

Ajustar o `tsconfig.json`

Abra o arquivo `tsconfig.json` e confirme estas opções:

json

```
{  
  "compilerOptions": {  
    "target": "ES2020",  
    "module": "commonjs",  
    "rootDir": "./src",  
    "outDir": "./dist",  
    "strict": true  
  }  
}
```



Typescript



Criar as pastas e o arquivo TypeScript

Crie a pasta `src`:

```
mkdir src
```




Typescript



Crie o arquivo `src/index1.ts` com este código:



Ty

```
class Pessoa {  
  nome: string;  
  idade: number;  
  
  constructor(nome: string, idade: number) {  
    this.nome = nome;  
    this.idade = idade;  
  }  
  
  apresentar(): string {  
    return `Olá! Meu nome é ${this.nome} e tenho ${this.idade} anos.`;  
  }  
}  
  
const aluno1 = new Pessoa("Ana", 20);  
console.log(aluno1.apresentar());
```



Typescript

Compilar o código

No terminal:

```
npx tsc
```

Isso gera o arquivo compilado em
`dist/index1.js`.





Typescript

Executar com Node

Agora rode:

```
node dist/index1.js
```

Saída esperada:

```
Olá! Meu nome é Ana e tenho 20 anos.
```





Typescript

Exercício em JavaScript (sem TS)

Arquivo: `pessoa.js`





Ty

```
class Pessoa {
  constructor(nome, idade) {
    this.nome = nome;
    this.idade = idade;
  }

  apresentar() {
    return `Olá! Meu nome é ${this.nome} e tenho ${this.idade} anos.`;
  }
}

const aluno1 = new Pessoa("Ana", 20);

// Aqui criamos um erro proposital: passamos a idade como string
const aluno2 = new Pessoa("Carlos", "vinte");

// Roda sem reclamar
console.log(aluno1.apresentar());
console.log(aluno2.apresentar());
```





Typescript



Saída no console:

Olá! Meu nome é Ana e tenho 20 anos.

Olá! Meu nome é Carlos e tenho vinte anos.



Typescript

Exercício em TypeScript (com TS)

Arquivo: `src/pessoa.ts`





Type

```
class Pessoa {  
  nome: string;  
  idade: number;  
  
  constructor(nome: string, idade: number) {  
    this.nome = nome;  
    this.idade = idade;  
  }  
  
  apresentar(): string {  
    return `Olá! Meu nome é ${this.nome} e tenho ${this.idade} anos.`;  
  }  
}  
  
const aluno1 = new Pessoa("Ana", 20);  
  
// Aqui criamos o mesmo erro: idade como string  
const aluno2 = new Pessoa("Carlos", "vinte");  
  
console.log(aluno1.apresentar());  
console.log(aluno2.apresentar());
```





Typescript



O que acontece ao compilar com **npx tsc**:

error TS2345: Argument of type 'string' is not assignable to parameter of type 'number'.

O TypeScript **bloqueia antes de rodar** e avisa:

"Ei, você está passando **string** onde deveria ser **number**!"



Typescript



Pergunta 1: O JavaScript deixou rodar — isso pode causar bugs escondidos?

Pergunta 2: O TypeScript impediu o erro — isso ajuda em projetos grandes?

Pergunta 3: Em sistemas de banco, saúde ou educação, qual seria o risco de não validar tipos?



Typescript



Pergunta 1: O JavaScript deixou rodar — isso pode causar bugs escondidos?

Sim.

Como o **JavaScript não valida tipos**, o código roda mesmo quando você passa valores incorretos. Isso pode gerar **bugs silenciosos**, difíceis de descobrir.

Exemplo: você espera que “idade” seja um número para calcular algo, mas alguém passa **"vinte"** (string). O programa pode quebrar em outro ponto ou gerar um resultado errado sem avisar.



Typescript



Pergunta 2: O TypeScript impediu o erro — isso ajuda em projetos grandes?

Com certeza.

Em projetos grandes, com muitas pessoas trabalhando juntas, é **fácil cometer erros de digitação, confundir tipos ou usar dados errados**.

O **TypeScript age como um fiscal**, verificando tudo antes de rodar e evitando que erros simples se espalhem.

Isso **economiza tempo** (menos bugs em produção) e **dá mais confiança** no código.



Typescript

Pergunta 3: Em sistemas de banco, saúde ou educação, qual seria o risco de não validar tipos?

O risco é **grave**, porque dados errados podem causar **consequências reais**:

- **Banco:** transferir valores incorretos, somar números com textos e perder dinheiro.
- **Saúde:** calcular dosagem de medicamentos errada porque recebeu "duas" em vez de 2.
- **Educação:** registrar notas como "dez" em vez de 10, alterando médias e aprovações.

Em áreas críticas, **não validar tipos pode gerar prejuízos financeiros, problemas legais ou até colocar vidas em risco.**





Lista de Exercícios

1. Classe Simples – Pessoa

Crie uma classe **Pessoa** com os atributos **nome** (string) e **idade** (number).

- Implemente o método **apresentar()** que retorna uma frase de apresentação.
- Crie 2 objetos de **Pessoa** e imprima o resultado.





Lista de Exercícios

1. Classe Simples – Pessoa

Crie uma classe **Pessoa** com os atributos **nome** (string) e **idade** (number).

- Implemente o método **apresentar()** que retorna uma frase de apresentação.
- Crie 2 objetos de **Pessoa** e imprima o resultado.





Lista de Exercícios

2. Animais – Polimorfismo

Crie a classe `Animal` com o método `falar()`.

Depois crie `Cachorro` e `Gato` que sobrescrevem `falar()`.

- Coloque os animais em um array do tipo `Animal[]`.
- Percorra o array e imprima o som de cada um.





Lista de Exercícios

3. Produto com Desconto

Crie uma classe **Produto** com **nome** e **preço**.

- Adicione o método **desconto(porcentagem)** que calcula o valor com desconto.
- Teste com um produto de R\$ 200 e desconto de 10%.

